LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 : RELASI ANTAR KELAS

NAMA : Wahyuni Salsabila

NIM : 103132400010

KELAS : S1 SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam pemrograman berorientasi objek, sebuah sistem yang kompleks tidak mungkin hanya dibangun dari satu class yang berdiri sendiri. Berbagai class perlu saling terhubung, bertukar informasi, atau membentuk kesatuan fungsi tertentu melalui apa yang disebut relasi antarkelas. Relasi ini bertujuan untuk memodelkan bagaimana objek-objek di dunia nyata saling berinteraksi mulai dari sekadar interaksi sesaat, hubungan kepemilikan yang fleksibel, hingga ketergantungan hidup-mati yang sangat erat.

1. Diagram Kelas (Class Diagram)

   Sebelum menuangkan rancangan ke dalam kode program, struktur class dan relasinya biasanya dipetakan terlebih dahulu menggunakan diagram kelas (UML). Sebuah class digambarkan dalam kotak bertingkat tiga: nama class di bagian paling atas, atribut di bagian tengah, dan method di bagian bawah. Untuk mendukung prinsip enkapsulasi, visibilitas setiap anggota ditandai dengan simbol akses, yaitu tanda plus (+) untuk *public*, minus (-) untuk *private*, dan tagar (#) untuk *protected*. Hubungan antarkelas kemudian digambarkan melalui garis penghubung dengan simbol ujung yang berbeda-beda sesuai jenis relasinya.

2. Asosiasi

   Asosiasi adalah bentuk hubungan paling dasar dan fleksibel antara dua class yang sebenarnya saling mandiri. Hubungan ini terjadi ketika suatu objek butuh berkomunikasi atau meminjam fungsi dari objek lain tanpa harus memilikinya secara permanen sebagai atribut. Interaksi asosiasi umumnya berlangsung sesaat, misalnya ketika suatu objek dilewatkan sebagai parameter di dalam sebuah method. Karena tidak ada kepemilikan permanen, siklus hidup kedua objek ini benar-benar terpisah; jika salah satu objek selesai digunakan atau dihapus, objek lainnya tetap ada dan tidak terpengaruh sama sekali.

3. Agregasi

   Agregasi merupakan hubungan kepemilikan bertipe *has-a* (bagian dan keseluruhan / *whole-part*), namun dengan ikatan yang tergolong longgar. Dalam agregasi, class wadah menampung objek lain sebagai anggotanya (misalnya dikumpulkan di dalam array), tetapi objek-objek anggota tersebut diciptakan secara independen di luar class wadah sebelum dimasukkan. Ciri khas utama agregasi adalah siklus hidup objek yang tetap mandiri: apabila objek penampungnya dibatalkan atau dihancurkan, objek-objek yang ada di dalamnya tidak ikut musnah dan masih bisa digunakan untuk keperluan lain. Pada diagram kelas, relasi ini digambarkan dengan garis berujung belah ketupat (wajik) kosong pada sisi class wadah.

4. Komposisi

   Komposisi merupakan bentuk khusus dari agregasi dengan tingkat keterikatan kepemilikan yang sangat kuat dan ketat. Pada relasi ini, bagian internal suatu objek tidak bisa berdiri sendiri di luar objek utamanya. Objek bagian biasanya

langsung dibuat (*di-instansiasi*) menggunakan kata kunci new di dalam *constructor* class pemiliknya, bukan dikirim dari luar. Akibatnya, siklus hidup objek internal terikat penuh pada objek utamanya: jika class pemiliknya dihapus atau musnah dari memori, seluruh komponen bagian di dalamnya otomatis akan ikut musnah. Pada diagram kelas, komposisi disimbolkan dengan garis berujung belah ketupat (wajik) berwarna solid/hitam penuh pada sisi class pemiliknya.

B. Guided

I. Guided 01

1. Mahasiswa.java

```
package modul5caca.guided1;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) { //ini setter nya
        this.nilai = nilai;
    }
    public int getNilai() { //ini getter nya
        return this.nilai;
    }
    public String getNama() {
        return this.nama;
    }

 public String getNim(){
 return this.nim;
}
}
```

2. Dosen.java

```
package modul5caca.guided1;

public class Dosen {
    private String kodeDosen;
```

```
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
a
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi-antar-kelas\project
\caca\src\main\java> java modul5caca.guided1.Dosen
Nim :130118383
Nama :Budi
Nilai :90
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi-antar-kelas\project
\caca\src\main\java>
```

II. Guided 02 package agregasi

1. Jurusan.java

```java
package modul5caca.guided.agregrasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, NamaJurusan;
    private Mahasiswa[] daftarMahasiswa= new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String NamaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.NamaJurusan = NamaJurusan;
    }
    //behaviour
    public void tambahMahasiswa(Mahasiswa m){
        daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }
    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " +
this.kodeJurusan);
        System.out.println("Nama Jurusan: " +
this.NamaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i < jumlahMahasiswa; i++){
            System.out.println(i+1 + ". " +
daftarMahasiswa[i].getNIM() + " - " +
daftarMahasiswa[i].getNama());

        }
    }
    //getter
    public String getKodeJurusan() {
        return this.kodeJurusan;
    }
    public String getNamaJurusan() {
        return this.NamaJurusan;
    }
    //setter
```

```
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }
    public void setNamaJurusan(String NamaJurusan) {
        this.NamaJurusan = NamaJurusan;
    }

}
```

2. **Mahasiswa.java**

```
package modul5caca.guided.agregrasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) { //ini
constructor
        this.NIM = NIM;
        this.Nama = Nama;
    }
    //getter
    public String getNIM() {
        return this.NIM;
    }
    public String getNama() {
        return this.Nama;
    }
    //setter
    public void setNIM(String NIM) {
        this.NIM = NIM;
    }
    public void setNama(String Nama) {
        this.Nama = Nama;
    }
}
```

Output :

```
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi-antar-kelas\project
\caca\src\main\java> java modul5caca.guided.agregrasi.Jurusan
Kode Jurusan: IF01
Nama Jurusan: Informatika
Daftar Mahasiswa:
1. 22001 - Caca
2. 22002 - Andi
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi-antar-kelas\project
\caca\src\main\java>
```

III. Guided 02 package komposisi

1. Komputer.java

```java
package modul5caca.guided.komposisi;
public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
    public static void main(String[] args) {
        Komputer pc = new Komputer("Intel Core i7", 3200, "LG
UltraGear", "Logitech", "Wireless");

        pc.displaySpecKomputer();
    }
}
```

2. Monitor.java

```java
package main.java.com.caca.guided.guided02.hargatoken;

public class hargaToken {
    public void info() {
        System.out.println("Harga Token");
```

```
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

3. Mouse.java:

```
package modul5caca.guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

4. Processor.java

```
package modul5caca.guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
```

```
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output :

```
Komputer.java
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi-antar-kelas\project
\caca\src\main\java> java modul5caca.guided.komposisi.Komputer
=== SPESIFIKASI KOMPUTER ===
Intel Core i7, 3200
LG UltraGear
Logitech, Wireless
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi-antar-kelas\project
\caca\src\main\java>
```

C. Unguided

1. Soal Unguided

Jawaban :

- RincianHarga.java

```java
package modul5caca.unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    // Constructor
    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10; // Pajak 10%
    }

    // Method hitung total
    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }

    // Getter
    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }
```

```
    // Setter
    public void setHargaDasar(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public void setPajak(double pajak) {
        this.pajak = pajak;
    }
}
```

- Pesanan.java

```
package modul5caca.unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    // Constructor
    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Kapasitas pesanan sudah
penuh!");
        }
    }
    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            System.out.println((i + 1) + ". " +
item.getNamaItem() + " - Rp " +
item.getRincian().getHargaTotal()
                    + " (Harga: " +
item.getRincian().getHargaDasar() + ", Pajak: " +
item.getRincian().getPajak() + ")");
```

```
        }
    }

    // Getter
    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }

    // Setter
    public void setNomorMeja(int nomorMeja) {
        this.nomorMeja = nomorMeja;
    }

    public void setDaftarItem(ItemPesanan[] daftarItem) {
        this.daftarItem = daftarItem;
    }

    public void setJumlahItem(int jumlahItem) {
        this.jumlahItem = jumlahItem;
    }
}
```

- ItemPesanan.java

```
package modul5caca.unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian; // Objek internal
komposisi

    // Constructor (Komposisi: RincianHarga di-instansiasi
di dalam sini)
    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    // Getter
    public String getNamaItem() {
        return this.namaItem;
```

```
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }

    // Setter
    public void setNamaItem(String namaItem) {
        this.namaItem = namaItem;
    }

    public void setRincian(RincianHarga rincian) {
        this.rincian = rincian;
    }
}
```

Main.java :

```
package modul5caca.unguided;

public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 = new ItemPesanan("Iced Americano",
18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant",
25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte",
22000);

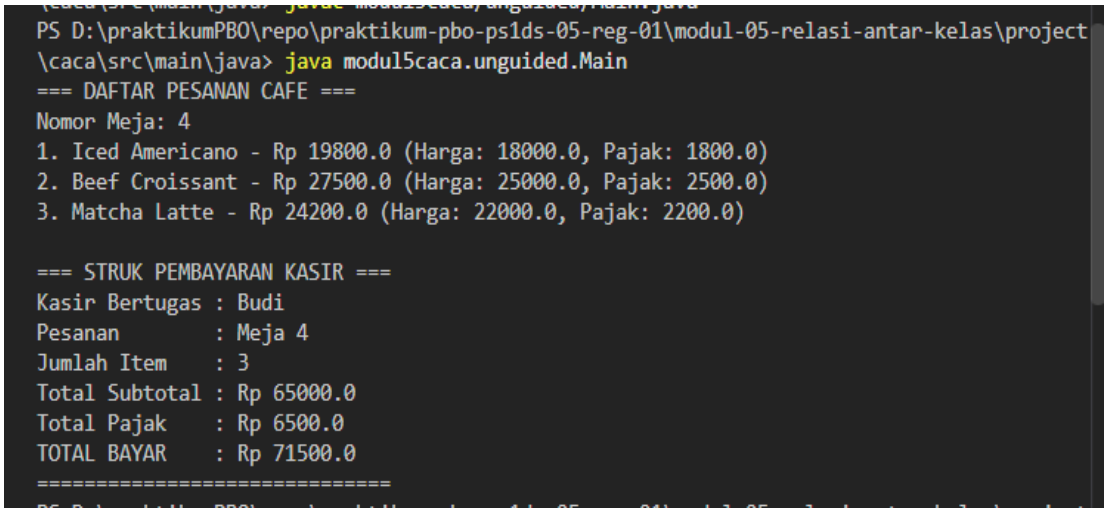
        Pesanan pesanan = new Pesanan(4, 5);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        System.out.println();

        kasir.prosesPembayaran(pesanan);
    }
}
```

Output: :



Penjelasan :

Program ini aku rancang sebagai sistem manajemen pesanan cafe sederhana yang mengelola alur pemesanan menu pelanggan hingga proses pencetakan struk pembayaran oleh kasir. Fokus utama di modul ini adalah membedakan tiga jenis relasi antarkelas dalam PBO. Pertama, aku menerapkan **komposisi** pada relasi antara ItemPesanan dan RincianHarga, di mana objek RincianHarga langsung di-instansiasi (new) di dalam *constructor* milik ItemPesanan. Hal ini bikin siklus hidup rincian harga terikat penuh; kalau item pesanannya dihapus, rincian harganya otomatis ikut hilang. Kedua, aku menerapkan **agregasi** pada Pesanan dan ItemPesanan. Di sini objek ItemPesanan dibuat secara mandiri di luar, lalu ditampung ke dalam array daftarItem lewat method tambahItem(), sehingga secara konsep item menu tetap bisa berdiri sendiri meski pesanan dibatalkan. Ketiga, relasi **asosiasi** diaplikasikan antara Kasir dan Pesanan, di mana class Kasir sama sekali tidak menyimpan objek Pesanan sebagai atribut permanen, melainkan hanya berinteraksi sesaat lewat parameter di method prosesPembayaran(). Selain itu, prinsip **enkapsulasi** juga diterapkan penuh di semua kelas dengan mengunci seluruh atribut menjadi private dan memakai kata kunci this secara eksplisit pada *constructor* dan *setter*.

Alur programnya sendiri berjalan terstruktur di class Main. Pertama, program membuat satu objek Kasir bernama Budi, lalu membuat tiga objek ItemPesanan terpisah *("Iced Americano", "Beef Croissant", dan "Matcha Latte")* lengkap dengan harga dasarnya masing-masing di mana nominal pajak 10% dan harga akhir langsung dihitung otomatis di balik layar oleh kelas RincianHarga. Setelah itu, dibuat satu objek Pesanan untuk Meja 4 dengan kapasitas array 5 slot, lalu ketiga menu dimasukkan satu per satu ke dalam pesanan tersebut. Program kemudian memanggil

tampilkanPesanan() untuk mencetak daftar menu yang dipesan beserta rincian harganya. Di tahap akhir, objek Kasir menjalankan method prosesPembayaran(pesanan) untuk mengiterasi seluruh item di dalam pesanan, mengakumulasikan total subtotal, total pajak, dan total bayar, hingga akhirnya mencetak struk pembayaran kasir yang rapi ke terminal.

D. Kesimpulan

Dari praktikum modul ini, dapat disimpulkan bahwa relasi antarkelas memungkinkan kita memodelkan interaksi dunia nyata ke dalam program secara lebih realistis dan terstruktur dengan membedakan tingkat keterikatan serta siklus hidup masing-masing objek. Kita dapat mengidentifikasi bahwa komposisi memiliki keterikatan hidup-mati yang mutlak karena objek internalnya langsung dibuat di dalam *constructor* induknya (seperti ItemPesanan dengan RincianHarga), agregasi memberikan relasi kepemilikan yang lebih longgar karena objek anggotanya diciptakan secara mandiri sebelum ditampung ke dalam class wadah (seperti Pesanan yang menampung ItemPesanan), dan asosiasi menjadi bentuk interaksi paling bebas di mana dua objek independen saling memanfaatkan fungsi hanya lewat parameter method tanpa saling memiliki secara permanen (seperti Kasir saat memproses Pesanan). Dengan memahami perbedaan ketiga relasi ini dan memetakannya lewat diagram kelas serta prinsip enkapsulasi, kita bisa membangun arsitektur kode Java yang jauh lebih rapi, modular, dan tidak menimbulkan ketergantungan antarkomponen yang berlebihan.